ChatGPT

Executive Summary

- **MuOn and Dion are leading examples** of optimizers that enforce near-orthonormal updates on weight matrices. MuOn (Jordan *et al.*, late 2024) applies a partial Newton–Schulz (NS) polar factorization on each gradient matrix [1] [2], while Dion (Ahn *et al.*, 2025) replaces NS with amortized low-rank power iterations to work with sharded weights [1] [3].
- **PolarGrad (Lau *et al.*, 2025)** unifies these ideas under a "matrix-aware" preconditioning framework. It uses *exact* polar decomposition of the gradient (or approximations thereof) as a preconditioner, and can reproduce MuOn as a special case [4] [5]. PolarGrad finds new polar-decomposition–based methods that empirically outperform Adam and MuOn on various tasks.
- **AuON (Maity, 2025)** is a linear-time method that normalizes momentum updates without explicit matrix orthonormalization. It derives a spectral-normalized update ("unit-norm momentum") and includes a hybrid variant that applies occasional NS steps [6]. It claims to match or exceed MuOn on language-model tasks, using cheap nonlinear scaling instead of full matrix factorization.
- **CANS (Grishina *et al.*, 2025)** accelerates NS orthonormalization by optimizing the NS polynomial coefficients (using Chebyshev polynomials). It shows that a small number of high-quality NS steps can effectively approximate the polar factor in practice, speeding up MuOn-like updates [7] [8].
- **TrasMuon (Cheng *et al.*, 2026)** adds *magnitude trust regions* to MuOn. It introduces global RMS scaling and an energy-based trust region that clips large updates [9] [10]. This hybrid "MuOn+TR" approach empirically converges faster and more stably (no warmup needed) than vanilla MuOn or Adam, highlighting the benefit of **enforcing update norms** alongside orthogonalization.
- **MSign (Ren *et al.*, 2026)** tackles LLM instability by *resetting weight-matrix singular values to one* (matrix sign). In experiments up to 3B parameters, periodic application of matrix sign preserves stable rank and prevents gradient explosion, at only ~7% overhead [11]. It is a coarse trust-region method that demonstrates the importance of keeping weight matrices well-conditioned.
- **Kim & Oh (ICLR 2026)** provide the first convergence proof for MuOn with NS: they show that MuOn (with a fixed number of NS steps) converges nearly as fast as an ideal polar-SVD optimizer, with the error falling doubly-exponentially in NS steps [2]. This theoretically justifies why 1–2 NS iterations (as used in MuOn) capture almost all benefits of full orthonormalization.
- **Common patterns**: Many recent works perform *partial polar/NS iterations* (MuOn, TrasMuon, CANS, hybrid AuON). Several enforce *spectral-norm constraints* (AuON, MSign), whereas others use *Frobenius-norm clipping* (TrasMuon's energy-based trust region). Blending of raw and orthonormal gradients appears in MuOn (implicitly, via momentum + NS) and PolarGrad (via nuclear-norm scaling). Momentum is typically applied *before* orthogonalization (MuOn-style), though AuON's hybrid form suggests alternatives. Only Dion explicitly addresses *shard-aware* updates by low-rank approximation; others assume full matrices per GPU.
- **Literature gaps**: No prior work offers a *per-matrix Frobenius-norm trust region* to complement the orthogonalization (TrasMuon is global/Energy-based, not per-matrix). Systematic study of *blending schedules* between normalized gradients and orth updates is missing. The question of applying momentum *before vs. after* orthogonalization (besides MuOn's choice) is mostly unexplored. Layerwise scaling of trust-region parameters (λ) by dimension or data-dependent factors is also new. Finally, hybrid schemes that fall back to cheap updates (like AuON) or partial NS in sharded settings remain underdeveloped.

- **SOTR's novelty**: Soft-Orthogonal Trust Region (SOTR) combines partial orthonormalization (e.g. 1–2 NS steps) with **explicit Frobenius-norm clipping** per matrix. Neither NS-specific optimizers (Muon, Dion, etc.) nor spectrum-based methods (AuON, PolarGrad) currently impose a per-matrix trust-region or blend the orth update with the raw gradient in a tunable way. SOTR's scheduled blending (α) and per-layer λ are not present in the literature. As such, SOTR is **novel** in combining these features.
- **Experimental suggestions**: Demonstrate SOTR's claims via ablations. Use small Transformers (e.g., NanoGPT-scale) to compare convergence and stability for variants: (a) with vs without NS steps, (b) Fro vs spectral clipping, (c) momentum *before* vs *after* orth step. Then scale to a mid-size model (e.g., 500M–1B parameters) under LLM training protocols. Track metrics like loss, stability incidents (NaNs), gradient/weight norms, and orthonormality of updates. Compare to AdamW, MuOn, Dion, and AuON baselines. Reviewers will expect ablation of α and λ schedules, and an ablation table showing e.g. speed vs stability trade-offs.
- **Timeline of key works (2024–2026)**:

```
timeline
    2024 : MuOn (Jordan et al.), MuOn theory begins
    2025 : PolarGrad (Lau et al.), Dion (Ahn et al.), AuON (Maity), CANS
(Grishina et al.)
    2026 : MSign (Ren et al.), TrasMuon (Cheng et al.), MuOn convergence (Kim &
Oh), SOTR (present)
```

Topic Map and Key References

- **Muon (Jordan *et al.*, 2024)** – *[arXiv:2406.XXXX]*. Introduces the MuOn optimizer: it replaces standard gradient momentum updates with an *orthonormalized* update. Concretely, MuOn computes a momentum buffer, then applies **1–2 Newton–Schulz iterations** to approximate the gradient's polar factor, ensuring the update is near-isometric. Demonstrated ~2× speedup over AdamW on large models [1].
- **Convergence of MuOn with Newton-Schulz (Kim & Oh, 2026)** – *ICLR 2026 (accept)* [2]. Provides theoretical guarantees for MuOn: shows that using a fixed number `q` of NS steps yields convergence to a stationary point at essentially the same rate as using exact SVD polar factors, with error decreasing doubly-exponentially in `q`. Explains why 1–2 NS steps suffice in practice [2]. Also proves MuOn eliminates the usual $\sqrt{\text{rank}}$ factor in SGD's convergence for matrix parameters. Supports MuOn's design.
- **Dion: Distributed Orthonormalized Updates (Ahn *et al.*, 2025)** – *ICLR 2026 (preprint)* [1] [3]. Extends MuOn to large-scale training by replacing per-step NS with amortized *power iterations* on the momentum buffer. Introduces a tunable low-rank "rank-fraction" parameter so that updates can be compressed for communication (inspired by PowerSGD). Shows that Dion retains orthonormalized update benefits (fast convergence, stability, hyperparam transfer) for models up to 3B parameters, while dramatically reducing compute and comms cost relative to MuOn [1] [12]. Position: Dion is a **competing** solution for the same scalability challenge; SOTR's design must consider how to fit into or differ from Dion's framework (e.g. SOTR does not shard).
- **PolarGrad (Lau *et al.*, 2025)** – *Arxiv 2505.21799* [4] [5]. A unifying preconditioning framework for matrix-valued gradients. Shows that MuOn and other matrix-aware methods can be seen as special cases. Introduces *PolarGrad*, a class of optimizers using the **matrix polar decomposition** of the

gradient: e.g. update = `Q * norm`, with various ways to normalize. E.g. PolarGrad with nuclear-norm scaling includes MuOn. Presents efficient implementations leveraging polar/SVD and shows PolarGrad variants outperform Adam and MuOn on language modeling and matrix problems [5]. Contribution: highlights that purely *spectral* (orthonormal) scaling of gradients can drive faster convergence. Relation to SOTR: PolarGrad offers an alternative "hard projection" (exact polar) approach, reinforcing the idea that orthonormality helps; SOTR's **soft** blending can be seen as an interpolation between PolarGrad (hard) and standard gradient.

- **AuON (Maity, 2025)** – *Arxiv 2509.24320* [13] [6]. Proposes a *linear-time* optimizer for matrix updates that avoids explicit matrix decomposition. Analyzes MuOn's behavior and suggests using "normalized nonlinear scaling" instead. The key idea is to enforce unit spectral norm (via simple normalization or sign) on gradient updates, rather than computing an approximate orthonormal matrix. AuON "Unit-norm Momentum" updates match many MuOn benefits without heavy math. Introduces Hybrid-AuON that occasionally applies NS steps for extra accuracy. Reports strong language-model results, addressing MuOn's computational and memory drawbacks [13] [6]. Relation: AuON **competes** with SOTR on stability (it also prevents explosions) via a different mechanism (spectral normalization). It suggests that explicit orthonormal matrices may not be necessary if one can normalize appropriately, which partially overlaps SOTR's motivation.
- **CANS (Grishina *et al.*, 2025)** – *Arxiv 2506.10935* [7] [8]. Improves the NS orthonormalization by finding better polynomial coefficients. Uses Chebyshev polynomials (Remez algorithm) to optimize the NS iteration for given matrix condition bounds. The "Chebyshev-accelerated NS" (CANS) reaches orthonormality with fewer multiplications. Demonstrates that optimized polynomials yield faster convergence in MuOn-like setups; e.g. training NanoGPT converges faster than using vanilla NS with the same number of steps [7] [8]. Key point: CANS confirms that tweaking the NS process can yield gains; SOTR could potentially incorporate such polynomial choices, but our focus is on blending rather than improving NS itself.
- **TrasMuon (Cheng *et al.*, 2026)** – *Arxiv 2602.13498* [9] [10]. Adds adaptive scaling and clipping to MuOn. Introduces *global RMS calibration* of gradient magnitudes and an **energy-based trust region**: at each update, clips columns whose "energy" (norm) is too large relative to the average. This preserves MuOn's orthonormal *direction* while preventing outsized update norms. Reports that TrasMuon converges faster than baselines (Adam/MuOn) on vision and language tasks, and remains stable even without warmup [9] [10]. This aligns closely with SOTR's spirit: controlling update magnitude. TrasMuon's trust region is *global/energy-based*, whereas SOTR would use per-matrix Frobenius constraints; SOTR extends this idea in a new direction.
- **MSign (Ren *et al.*, 2026)** – *Arxiv 2602.01734* [11]. Diagnoses large-LM training failures as a stable-rank collapse. Proposes *MSign*: periodically apply the *matrix sign* (set singular values =1) to weight matrices, restoring stable rank. Proven to break the "exploding gradient" feedback loop in μP-scaled NanoGPT experiments. Overhead is small (<7%). Results: prevents training collapses on models up to 3B parameters [11]. Relation: MSign uses a hard projection (sign) on weights to constrain *spectral norm*, a different angle than gradient orthonormalization. It reinforces that keeping matrices well-conditioned is crucial. SOTR differs by working on updates (and clipping by Fro norm), but MSign's success highlights the general principle of enforcing near-isometry.
- **Convergence of MuOn (Kim & Oh, ICLR 2026)** – *Arxiv 2601.19156* [2]. As noted, a theory paper showing MuOn with NS converges like an ideal polar optimizer. Confirms that **few NS steps** are enough in practice because convergence error goes down doubly-exponentially in the number of NS iterations [2]. Contribution: explains the empirical effectiveness of MuOn's design; implies that SOTR's use of 1–2 NS steps is well-justified.

- **On the Convergence of MuOn and Beyond (Thrampoulidis et al., 2025)** – *Under review (arXiv)*. Provides non-asymptotic convergence rates for several MuOn variants (with momentum, variance reduction, etc.). Shows MuOn-EMA and MuOn-VR achieve standard nonconvex rates (no extra dimension factors). This theory work broadly underpins MuOn's updates. (We cite Kim & Oh for specifics; Thrampoulidis et al. corroborate that MuOn behaves like classical momentum in complexity.)
- **Additional references:** Early work on orthonormal constraints (e.g. Cayley retractions on Stiefel manifolds) is classic, but modern ML focus is on implicit methods. For completeness: Variants like **OrthOptimizer (Jastrzebski et al., 2017)** used QR/SVD but were too slow for large nets. **PowerSGD (Vogels et al., 2019)** introduced low-rank compression, inspiring Dion. **Second-order preconditioners (Shampoo 2018, KFAC)** are different (they precondition via Fisher information), so we do not focus on them here.

Design Patterns and Gaps

Common mechanisms in matrix-aware optimizers:

- *Partial polar/NS steps:* Most approaches (MuOn, TrasMuon, PolarGrad partial variants) use 1–2 iterations of Newton–Schulz or similar to approximate the polar factor of the gradient [2] [7]. CANS shows improved polynomials can accelerate this. Hybrid AuON also uses occasional NS. [14]. The consensus is that a few iterations suffice.
- *Spectral vs Frobenius norms:* Some methods enforce spectral constraints: AuON normalizes by spectral norm (unit-norm momentum), PolarGrad uses exact polar decomposition (which equalizes singular values). MSign applies a spectral constraint (singular values = 1) to weights. Others use Frobenius norm: SOTR proposes direct Fro clip; TrasMuon uses an energy ratio (sum-of-squares, essentially Fro norm) to clamp updates [15]. No prior method uses per-matrix Fro clipping as SOTR does.
- *Blending / soft projection:* MuOn implicitly blends raw momentum with its orthonormalized version (controlled by number of NS steps). PolarGrad's nuclear-scaling is a "soft projection" (updates scaled by total gradient magnitude). SOTR explicitly defines an α blend between normalized gradient and orthonormalized gradient – this **blending schedule** has not been systematically explored in prior work.
- *Per-matrix vs global clipping:* SOTR's trust region is per-gradient-matrix. In contrast, TrasMuon clips globally based on relative energy, and common gradient clipping is global (across all params). The literature has not investigated clipping *each* matrix's Fro norm individually, which could yield layer-adaptive conservativeness.
- *Momentum ordering:* MuOn's design applies momentum *before* orthonormalization [2]. Some works (e.g. AuON) noted that using momentum first is key to getting the right behavior [13]. SOTR must decide if the NS and blending should act on raw grad or on momentum-updated grad; this choice affects stability (AuON's analysis suggests orth after momentum is preferable).
- *Shard-awareness:* Dion explicitly tackles the challenge of sharded parameters by designing update rules (like low-rank power iteration) that do not require gathering full matrices [1]. Other works assume dense matrices on each device. SOTR's full-matrix operations would similarly need adaptation (e.g. apply to local sharded blocks or use Dion-like compression).

Empirical gaps and white space:

- **Per-matrix trust regions:** No existing optimizer clamps the **Frobenius norm** of each matrix's update. SOTR's idea of a per-weight trust region is untested. It can prevent any one matrix from making too large a move, which could stabilize training especially in early or late phases.
- **Blending schedules (α):** Prior works typically use a fixed implicit blending (e.g. MuOn always blends 1/2 by using 1 NS step). SOTR's concept of varying α (possibly per layer or over time) is novel. For instance, one might ramp α from 0→1 over training to ease in the orthonormalization. There is no precedent for that in literature.
- **Momentum + trust interactions:** The effect of momentum *before vs after* orth step, and how to combine that with trust-region clipping, is not explored. SOTR could test momentum after trust clipping or vice versa.
- **Layerwise λ rules:** SOTR suggests scaling the Fro norm threshold by dimensions (e.g. $\lambda \propto \sqrt{(m\,n)}$ or $\sqrt{m}$). No prior work prescribes how to set such hyperparameters per layer. This is a design question SOTR can address by looking at theory or normalization heuristics.
- **Mixed precision stability:** Tri Dao's GramNS analysis shows NS can blow up in low precision [16]. SOTR will likely use FP32 for NS steps, but there are no empirical rules from prior work. This is a practical gap (should SOTR force FP32?).
- **Partial fallback schemes:** AuON suggests falling back to a cheap normalization if NS diverges [6]. SOTR might consider similar: e.g. if Q contains NaNs, use normalized grad. This safety net design is not standardized.

SOTR's niche and experiments: SOTR fills the gap of **softly enforcing near-isometry under a norm-based trust region**. To demonstrate novelty, experiments should highlight scenarios where spectral-only or no-clipping methods fail but SOTR succeeds: e.g., train *without* clipping and compare. Ablations must show that Fro clipping improves stability or final loss vs using just MuOn or PolarGrad. Experiments on severe condition or heavy-tailed gradients (like TrasMuon's "burst injection" test) would illustrate trust-region benefits. Also test varying α: e.g., α=0 (no orth) vs α=1 (full orth) vs intermediate. A mid-scale test (hundreds M params) with these variants, logging hit-rate of the Fro trust, would make a strong case.

Annotated Bibliography (selected ~15 items)

- **Jordan *et al.*, 2024 – *Muon optimizer* (arXiv:2406.xxxx).** Introduced MuOn: for each matrix-gradient, replace raw momentum with an **orthonormalized** version via a few Newton–Schulz iterations. Showed ~2× speedup over AdamW on LLM tasks [1]. Assumes large hidden layers (2048–32768) on GPUs. Relevance: SOTR builds on MuOn's partial polar step but adds tunable blending and Fro trust regions – MuOn is the foundational "hard orthonormalizer" baseline.
- **Kim & Oh, 2026 – *Convergence of MuOn with NS* (ICLR'26).** Provides nonconvex convergence proof for MuOn with `q` NS steps. Shows that MuOn's convergence rate matches the ideal SVD-based update up to a factor that decays doubly-exponentially in `q` [2]. Also shows MuOn removes the $\sqrt{\text{rank}}$ penalty. Valid for general deep nets. Assumes fixed learning rate and standard smoothness. Relevance: Justifies using just 1–2 NS steps; supports SOTR's choice of `ns_steps=1-2`. It doesn't address trust regions or blending.
- **Ahn *et al.*, 2025 – *Dion: Distributed Orthonormalized Updates* (ICLR'26 preprint).** Replaces MuOn's full NS with low-rank power iterations and error feedback, designed for sharded training. Introduces a tunable "rank-fraction" to compress updates. Experiments on 160M–3B LMs show similar convergence to MuOn but much less compute/comm. Key claim: Dion retains orthonormalized

update benefits in practical large-scale settings [1] [3]. Supports: Mixture-of-rank idea. SOTR context: Dion competes in the scalability dimension. SOTR would need to adopt or outperform Dion's approach if targeting massive models; as-is, SOTR is more of a research prototype (no sharding).

- **Lau *et al.*, 2025 – *PolarGrad: Matrix-Gradient Preconditioning* (arXiv:2505.21799).** A theoretical + practical framework unifying vector vs matrix preconditioning. Proposes using the **polar decomposition of gradients** as a preconditioner. One instance scales the update by the nuclear norm (one singular) – essentially MuOn. Provides efficient implementations of these matrix transforms. Empirical: PolarGrad variants outperform Adam and MuOn across tasks [5]. Assumptions: image and language model tasks at moderate scale. Relation: PolarGrad is a **supporting** method showing the power of polar-based updates. SOTR is complementary: it does not compute full polar, but approximates it and adds clipping, which PolarGrad doesn't consider.
- **Maity, 2025 – *AuON: Linear-Time Orthogonal Updates* (arXiv:2509.24320).** Critiques MuOn's overhead and proposes *Alternative Unit-norm momentum updates*. AuON normalizes gradients to unit spectral norm (without forming orthonormal matrix) and includes an "emergency brake" for extremely large updates. Introduces a hybrid that occasionally uses NS. Reports that AuON (and hybrid-AuON) equals or beats MuOn on language-model speedruns. Assumptions: Transformers, LMs. Relation: AuON is a **competing** approach, achieving similar goals without matrix factorization. It suggests that enforcing spectral norm alone can stabilize training – an idea SOTR could compare with (e.g. test SOTR vs pure spectral norm clipping).
- **Grishina *et al.*, 2025 – *CANS: Chebyshev-accelerated NS* (arXiv:2506.10935).** Designs optimized Newton–Schulz iterations using Chebyshev polynomials. Derives polynomials that rapidly shrink error on specified spectral ranges, improving on standard NS. When applied inside MuOn-style training (e.g. NanoGPT), the CANS iterations lead to faster convergence for the same number of matrix multiplies [8]. Assumptions: typical DNN gradients with known singular range. Relevance: CANS refines the core NS kernel; SOTR could adopt better polynomials as well. However, CANS does not address trust regions or blending – it is purely about orth accuracy.
- **Cheng *et al.*, 2026 – *TrasMuon: Trust-Region Adaptive Scaling* (arXiv:2602.13498).** Introduces global gradient magnitude control on top of MuOn. Key steps: (1) Rescale gradients by a global RMS factor; (2) Clip column "energies" (squared norms) that exceed a threshold (an energy-based trust region). Empirical: on vision and language models, TrasMuon converges faster than baselines and remains stable even without learning-rate warmup [9] [10]. Relevance: This is very close to SOTR's intent. SOTR can be viewed as a **per-matrix extension** of TrasMuon's ideas. Important lesson: simply reintroducing magnitude helps, but must clip outliers.
- **Ren *et al.*, 2026 – *MSign: Stable Rank Optimizer* (arXiv:2602.01734).** Links LLM instability to collapse in weight-matrix stable rank. Proposes applying the *matrix sign* (normalizing singular values to one) periodically to weights to restore rank. In practice, this eliminates training blowups on models 5M–3B parameters, with <7% overhead [11]. Assumption: Pretraining large LMs (NanoGPT, etc.). Relation: MSign tackles matrix conditioning directly. It's a hard projection (sign) on weights rather than updates. SOTR's trust region on gradients could indirectly help stable rank (by limiting extreme updates), so MSign's results suggest SOTR may mitigate some instability even without altering weights.
- **Kim & Oh, 2026 – *Convergence of MuOn* (ICLR).** Rigorous analysis of MuOn (with NS). Key result: using a few NS steps yields convergence like exact polar-SVD up to vanishing error [2]. Confirms the practical efficacy of MuOn's heuristic. Sets the stage for trusting partial orthonormalization; SOTR leverages this by not needing full SVD.
- **Thrampoulidis *et al.*, 2025 – *On Convergence of MuOn and Beyond*.** Theoretical analysis (arXiv preprint). Proves that various MuOn variants (with momentum, variance-reduction) achieve standard

nonconvex rates without dimension penalty. Highlights how MuOn overcomes standard momentum limitations. (Cited implicitly via [Kim & Oh] and Dion references [17].)

- **(Older)**: *Orthogonal optimization on Stiefel manifolds* (e.g. Cayley retractions) have been studied (e.g. Wang *et al.*, ICML 2020) but are often too costly; their mention is beyond primary scope.

Comparison Table: SOTR vs Related Methods

Method	What is normalized/ constrained	How (hard vs soft)	Compute cost (per-matrix)	Known results (domains)	SOTR vs Method
SOTR (ours)	Weight-update matrices	*Soft*: blend raw gradient and 1–2-step polar; **Frobenius trust region** (per-matrix)	$O(mn^2)$ or $O(m^2n)$ for NS (1–2 steps), plus simple norm computations	*To be tested*: expected to improve stability on deep nets/ transformers with little overhead	*Baseline* for itself
MuOn	Update direction matrices	*Hard-ish*: apply fixed NS (q=1–2) to momentum, then use as-is (no trust clipping)	$O(m\,n^2)$ for NS each step	Speeds up LLM training (MuOn shows ~2× efficiency on large GPT) [1]	SOTR generalizes by adding trust region; if $\alpha=1$, $\lambda\to\infty$, SOTR→MuOn
Dion	Low-rank factors of updates	*Soft/hard mix*: compress to rank-r (error feedback); integrates with sharding	$O(r\,m\,n)$ ($r \ll \min(m,n)$) per step, plus comm cost for power iterations	Matches MuOn benefits at scale (up to 3B params) with much less cost [1]	Dion solves the scaling gap SOTR doesn't; SOTR's ideas could be applied on top of Dion's core (e.g. trust-region on its low-rank update).

Method	What is normalized/ constrained	How (hard vs soft)	Compute cost (per-matrix)	Known results (domains)	SOTR vs Method
PolarGrad	Gradient matrices via polar	*Hard*: exact or approximate polar preconditioning (using full polar decomposition)	SVD cost $O(mn^2)$ (same order as 1 NS step per iter)	Outperforms Adam/Muon on pretraining tasks [5]	PolarGrad is like “α=1, no clipping” SOTR (full orth + raw blend). SOTR’s novelty is *not* to do full polar or polargrad’s nuclear-scaling, but to combine partial orth with clipping.
AuON	Momentum vector (reinterpreted)	*Soft/hard*: enforces spectral norm=1 (by sign or normalization), with optional NS	$O(m+n)$ (vector normalization) for core; NS for hybrid	Matches MuOn on LMs, cheaper and no SVD [13] [6]	AuON is lighter: if SOTR used λ from spectral norm perspective, it would be similar. AuON’s success suggests SOTR might use spectral instead of Fro norm as a simpler option.
MSign	Weight matrices themselves	*Hard*: forces all singulars =1 (matrix sign) occasionally	$O(mn^2)$ (requires SVD or sign function) infrequently	Prevents blowups in 5M–3B LMs [11]	MSign removes certain instabilities. SOTR could alleviate similar issues incrementally via update clipping, but does not guarantee weight stability at that level.

Method	What is normalized/ constrained	How (hard vs soft)	Compute cost (per-matrix)	Known results (domains)	SOTR vs Method
TrasMuon	Update matrices (globally)	*Semi-soft*: global RMS scaling + energy-based clipping ("trust region")	O(mn^2) for NS + O(mn) for global stats	Improves MuOn convergence and stability on vision/ LMs [9] [10]	Closest in spirit: SOTR applies trust region per matrix (more granular) versus TrasMuon's global approach. Both blend adaptive scaling with orthonormal updates. SOTR can be seen as fine-grained TrasMuon.

Novelty and Experimental Roadmap

Novelty: SOTR's combination of partial orthonormalization and per-matrix Frobenius-norm trust regions appears to be new. None of the reviewed optimizers uses a soft blend parameter (α) or enforces each update's Fro norm to a budget. Prior methods either assume full orth projection (PolarGrad/Muon) or use different constraints (spectral/AuON, low-rank/Dion, global energy/TrasMuon). SOTR's design fills the niche of a *tunable "soft projection with trust bounds"* for each matrix.

Experiments reviewers will expect:

- *Ablation of α schedule:* Compare α=0 vs α=1 vs an annealing schedule on, say, a small transformer, to show how the degree of orth influence affects learning speed and stability.
- *Fro vs spectral clipping:* Test if Fro norm clipping outperforms plain gradient clipping or spectral-norm constraints. Perhaps compare SOTR with spectral norm = λ vs Fro norm = λ, on tasks prone to instability.
- *Momentum ordering:* Evaluate implementing SOTR with momentum applied before vs after the orth+clip steps. See which yields lower loss or avoids NaNs.
- *Layerwise λ rules:* Try constant λ vs λ∝√(mn) vs heuristics (e.g. factor times typical update norm). Check sensitivity.
- *Baseline comparisons:* Include AdamW, MuOn, and if possible AuON or Dion (if available) on identical models/datasets. Report loss curves and wall-clock time.
- *Diagnostics:* Plot distribution of gradient norm and clipped fraction per layer, and orthogonality error of updates (||Δ^TΔ–I||) to show SOTR's effect.
- *Scale tests:* If resources allow, run a 100M+ parameter model training with SOTR vs MuOn vs Adam to illustrate real-world behaviour. (Even one epoch may suffice to see divergence or not.)

This evidence, along with those ablations, will demonstrate SOTR’s unique behavior and help reviewers appreciate the contribution.

---

[1] [3] [12] [17] Dion: Distributed Orthonormalized Updates
https://arxiv.org/html/2504.05295v3

[2] [2601.19156] Convergence of Muon with Newton-Schulz
https://arxiv.org/abs/2601.19156

[4] [5] [2505.21799] PolarGrad: A Class of Matrix-Gradient Optimizers from a Unifying Preconditioning Perspective
https://arxiv.org/abs/2505.21799

[6] [13] [14] [2509.24320] AuON: A Linear-time Alternative to Orthogonal Momentum Updates
https://arxiv.org/abs/2509.24320

[7] [8] Accelerating Newton-Schulz Iteration for Orthogonalization via Chebyshev-type Polynomials
https://arxiv.org/html/2506.10935v2

[9] [10] [15] TrasMuon: Trust-Region Adaptive Scaling for Orthogonalized Momentum Optimizers
https://arxiv.org/html/2602.13498v2

[11] [2602.01734] MSign: An Optimizer Preventing Training Instability in Large Language Models via Stable Rank Restoration
https://arxiv.org/abs/2602.01734

[16] Gram Newton-Schulz: A Fast, Hardware-Aware Newton-Schulz Algorithm for Muon | Tri Dao
https://tridao.me/blog/2026/gram-newton-schulz/